



OpenClaw in Docker + Ollama on Windows (WSL2 / Docker Desktop)

Stable setup using `host.docker.internal` + `gateway bind="lan"` (fixes `ERR_EMPTY_RESPONSE`)

Last revision: 11 Feb 2026

Goal

Run Ollama on Windows (localhost 127.0.0.1:11434) and OpenClaw inside Docker (via WSL2/Docker Desktop), connecting from the container to the Windows-hosted Ollama using `host.docker.internal`, with configuration stored in a Docker volume and Dashboard access from Windows via tokens.

Quick concept (why `host.docker.internal`)

OpenClaw in Docker is a separate environment (the container). To let it talk to Ollama running on Windows, we use the bridge hostname `host.docker.internal`. This keeps Ollama on localhost (not exposed to your LAN) while still allowing the container to reach it.

Security note

Everything here is local. However, the OpenClaw dashboard port (18789) is published on your host. Keep Windows Firewall enabled and do NOT forward this port on your router.

Prerequisites

- Windows 10/11 with WSL2 installed and an Ubuntu distro.
- Docker Desktop with WSL integration enabled.
- Ollama installed on Windows and responding at `http://127.0.0.1:11434`.
- Target model: `qwen3:8b` (CPU-only).
- OpenClaw dashboard port: 18789.

Minimum recommended hardware (CPU-only) for qwen3:8b

- Minimum RAM: 16 GB (works, but tight under multitasking).
- Recommended RAM: 32 GB (more stable).
- Free disk: 12–20 GB (model + cache + logs; varies by quantization).
- Larger contexts use more memory. This guide uses contextWindow=32768.

Models often cited as 'ideal' (requires better hardware)

If you have better hardware, these models are often mentioned in OpenClaw/Ollama materials. On CPU-only PCs, treat them as references—not immediate targets:

- qwen3-coder
- glm-4.7
- gpt-oss:20b
- gpt-oss:120b

In this guide, we stick with qwen3:8b because it's realistic for CPU-only systems.

Step-by-step (clean + stable install)

1) Verify model in Ollama (Windows PowerShell)

Use Windows (PowerShell) to check that Ollama responds and the model exists:

```
(Invoke-WebRequest -UseBasicParsing
http://127.0.0.1:11434/api/tags).Content
```

If you need to download the model:

```
ollama pull qwen3:8b
```

2) Create the project folder (WSL / Ubuntu)

```
mkdir -p ~/openclaw-docker
cd ~/openclaw-docker
```

3) Create docker-compose.yml

```
cat > docker-compose.yml <<'YAML'
services:
  openclaw-gateway:
    image: ghcr.io/openclaw/openclaw:latest
    container_name: openclaw-gateway
    ports:
      - "18789:18789"
```

```
volumes:
  - openclaw_data:/home/node/.openclaw
restart: unless-stopped
```

```
volumes:
  openclaw_data:
YAML
```

Port tip

If port 18789 is already in use, change the LEFT side of docker-compose.yml only. Example: 9000:18789. Then open <http://127.0.0.1:9000> in your browser.

4) Create openclaw.json (stable version)

Important points:

- Use api: "openai-responses" (Ollama provider with OpenAI-style compatibility).
- Set gateway.bind: "lan" to prevent ERR_EMPTY_RESPONSE (makes the gateway listen on 0.0.0.0 inside the container).
- Token below is just an example. Replace it with a strong token for real use.

```
cat > openclaw.json <<'JSON'
{
  "models": {
    "providers": {
      "ollama": {
        "baseUrl": "http://host.docker.internal:11434/v1",
        "apiKey": "ollama-local",
        "api": "openai-responses",
        "models": [
          { "id": "qwen3:8b", "name": "qwen3:8b", "contextWindow": 32768 }
        ]
      }
    }
  },
  "agents": { "defaults": { "model": { "primary": "ollama/qwen3:8b" } } },
  "gateway": {
    "mode": "local",
    "port": 18789,
    "bind": "lan",
    "auth": { "mode": "token", "token": "123456789" },
    "controlUi": { "enabled": true, "allowInsecureAuth": true }
  }
}
JSON
```

5) Start the container

```
docker compose up -d
```

6) Copy openclaw.json into the volume (correct path)

OpenClaw reads its config at /home/node/.openclaw/openclaw.json inside the container. Therefore, copy this file path into the volume:

```
docker run --rm -v openclaw-docker_openclaw_data:/home/node/.openclaw -v  
"$(pwd)/openclaw.json:/tmp/openclaw.json:ro" alpine sh -lc 'cp  
/tmp/openclaw.json /home/node/.openclaw/openclaw.json && ls -la  
/home/node/.openclaw/openclaw.json'
```

7) Fix permissions (prevents EACCES)

Create folders and adjust ownership/permissions before restart:

```
docker exec -u 0 openclaw-gateway sh -lc "mkdir -p  
/home/node/.openclaw/canvas /home/node/.openclaw/cron && chown -R  
node:node /home/node/.openclaw && chmod 644  
/home/node/.openclaw/openclaw.json"
```

8) Restart the container (required when changing bind)

Changes to gateway.bind require a restart so the gateway reopens on 0.0.0.0:

```
docker restart openclaw-gateway
```

Validation (3 quick checks)

A) Check status and ports (WSL)

```
docker ps --filter "name=openclaw-gateway" --format "table  
{{.Names}}\t{{.Status}}\t{{.Ports}}"
```

B) Confirm the gateway listens on 0.0.0.0 (WSL)

Use --since to avoid old logs:

```
docker logs --since 5m openclaw-gateway | grep -E "listening on|host  
mounted|Config invalid|Gateway auth"
```

Expected: something like "ws://0.0.0.0:18789" and/or "host mounted at
http://0.0.0.0:18789/..."

C) Confirm the container can reach Ollama on Windows (WSL)

```
docker exec openclaw-gateway sh -lc 'curl -sS  
http://host.docker.internal:11434/api/tags | head -c 200; echo'
```

D) Open the Dashboard on Windows

Open in your Windows browser:

```
http://127.0.0.1:18789/?token=123456789
```

Quick chat test: “What is the capital of Brazil?” (first response may be slower since the model will be starting up)

Troubleshooting (common issues)

1) ERR_EMPTY_RESPONSE / “127.0.0.1 didn’t send any data”

- Issue: the Windows browser shows ERR_EMPTY_RESPONSE and logs indicate the gateway is listening on 127.0.0.1 inside the container.
- Fix: ensure gateway.bind is "lan" (not an IP), copy the config into the volume path, then restart the container.
- Verify: logs in the last 5 minutes show ws://0.0.0.0:18789.

2) EACCES: permission denied (canvas/cron)

- Issue: logs show failures creating /home/node/.openclaw/canvas or /cron.
- Fix: re-run the permissions step (mkdir + chown) and restart the container.

```
docker exec -u 0 openclaw-gateway sh -lc "mkdir -p  
/home/node/.openclaw/canvas /home/node/.openclaw/cron    && chown -R  
node:node /home/node/.openclaw"  
  
docker restart openclaw-gateway
```

3) ‘Gateway auth is set to token, but no token is configured’

- Usually happens if the container started up before the JSON config was copied into the volume, or the config was not loaded yet.
- Fix: copy openclaw.json into /home/node/.openclaw/openclaw.json in the volume, then restart the container.
- Use docker logs --since 5m to confirm current state.

References

- Ollama: <https://ollama.com/>

- OpenClaw: Official project repository (GitHub) - <https://github.com/openclaw/openclaw>
- OpenClaw Ollama provider doc:
<https://github.com/openclaw/openclaw/blob/69aa3df116d38141626fcdc29fc16b5f31f08d6c/docs/providers/ollama.md>
- Ollama integrations: <https://docs.ollama.com/integrations/openclaw>

Document translated by AI and carefully reviewed by my daughter (Computer Engineering student).